Text to Knowledge by GPT2

A PREPRINT

Mingxuan Qu
Autonomous Systems
st181113@stud.uni-stuttgart.de
University of Stuttgart

Ling Xin
Autonomous Systems
st176945@stud.uni-stuttgart.de
University of Stuttgart

July 31, 2024

ABSTRACT

In this project, we generated a knowledge-graph-like dataset based on Wikidata and Wikipedia website, which contains text, triple sets, corresponding paragraphs, and hops. For training progress, the architecture of the GPT2 language model is applied. Our goal is to train this language model by using our constructed dataset so that this model is able to output related triple sets when a paragraph is given.
In order to accomplish this purpose, that is, to allow the language model to learn the input format correctly to generate accurate triples, a sufficiently large, well-constructed, and highly pure dataset plays a vital role. Therefore, we used the complete data officially provided by Wikidata website, and each part of our dataset was precisely differentiated by unique text separators.
From theory to implementation, we did extensive data cleaning works to ensure a good performance of pretraining of our language model.
Furthermore, the scoring method is essential. To better assess the final results, the F1-score is used as an evaluation criterion.

1 Introduction

Nowadays, knowledge graphs have become an crucial tool in the fields of data science and artificial intelligence, as they provide a structured approach to representing complex information through interconnected entities and relationships. Unlike traditional databases that store data in tables, knowledge graphs use graphical models that enable more comprehensive semantic queries and advanced reasoning. Popular applications such as Google Knowledge Graph have demonstrated that knowledge graphs can very effectively improve search accuracy and integrate different data sources.

There is a growing interest in cognitive graphs, which integrate and organize large amounts of disparate data to facilitate understanding and decision-making. By studying cognitive graphs, researchers and practitioners can develop intelligent systems that can solve complex data integration problems, improve information retrieval, and understand entities and their relationships.

Large-scale language models (LLMs) are a class of artificial intelligence models designed to understand and generate human language. They are usually trained on large amounts of textual data to capture the nuances and complexities of natural language. These models use deep learning techniques, particularly transform structures, to recognize patterns, context and meaning in text.

GPT2 is a typical example of a large-scale language model. It is based on a transformational architecture and is pre-trained with large amounts of text from the Internet. This extensive pre-learning allows GPT2 to learn a complete understanding of language, including grammar, world facts, and even logical skills. After this pre-learning phase, the GPT2 can be tuned to specific tasks, requiring less data and learning time for a particular task than a model trained from scratch.

model	params	train loss	val loss
gpt2	124M	3.11	3.12
gpt2-medium	350M	2.85	2.84
gpt2-large	774M	2.66	2.67
gpt2-xl	1558M	2.56	2.54

Table 1: GPT-2 baselines comparision

Table 1 presents various GPT models along with their respective parameters and performance metrics. It is evident that the performance improves progressively as the number of parameters increases.

The task of our lab project is to train the GPT2 language model and predict the corresponding ternary sets in it based on the input paragraphs or texts. Our dataset is clean and sufficiently broad which contains 1,503 text files. At there 37,689,686 different ternary sets which includes 11,339,043 unique entities and 596 unique relations were placed. After the tokenization of the dataset we obtained 33.2 billion tokens in total. Due to the enormous size of our dataset and the limited number of GPU resources available for the project, we were unable to use GPT models with an excessive number of parameters. In our case, we utilized the most basic GPT2 model with 124 million parameters. Even so, it took six days to complete a full training cycle of the model.

In our paper six sections are mainly involved: Introduction, Methodology, Implementation, Results, Conclusion and Future Works. The Introduction provides an structural overview of our knowledge-graph-like dataset and discusses the specific features of the experimental language models. In the Methodology section, methods which have been implemented on generating precise dataset and principles of GPT2 model are discussed in detail. The Implementation section is about Specific implementation process, problems encountered, and solutions. The Results section includes adjustments to the datasets and a comparative analysis of the results obtained. Next comes the conclusion section where the comprehensive summary of the entire lab project is provided. At last is the future works section where we give some outlook on parts that can continue to be optimized and improved in the future.

2 Methodology

Base on the goal of our task, we first need a specific dataset which order our training material in a particular way so that the language model can learn to generate the knowledge graph in textual format, namely relational triples, from the input text. Then we use Byte Pair Encoding (BPE) used by GPT2 to tokenize the dataset and train the GPT2 small model on it.

2.1 Set up dataset

Initially, there is no clean and suitable knowledge-graph-like dataset available. For our goal of task, we need to manually create the dataset. We planned to add four parts to our dataset: sentence, triples, paragraphs as well as chain of triples. For the triple part we extracted all the relational triples from wikidata dumps downloaded from wikidata website and formed them into correct triple format, each triple contains the label and ID of entities and relations, see table 2.

Subsequently, we constructed corresponding sentence based on each triple for the model to study how to generate knowledge graph triple from given text, see table 3.

Index	Head Entity	Relation	Tail Entity
Triple 1	[entity1, Q8]: happiness	[relation, P31]: instance of	[entity2, Q331769]: mood
Triple 2	[entity1, Q1]: universe	[relation, P361]: part of	[entity2, Q3327819]: multiverse

Table 2: Example of the extracted triples

Index	Sentence
Triple 1	Happiness is an instance of mood.
Triple 2	Universe is part of multiverse.

Table 3: Constructed sentences from triples in table 2

Entity	Triples
happiness	[entity1, Q331769]: mood, [relation, P31]: instance of, [entity2, Q8]: happiness, [relation, P31]: instance of, [entity3, Q9415]: emotion
universe	[entity1, Q1]: Universe, [relation, P793]: significant event, [entity2, Q323]: Big Bang, [relation, P31]: instance of, [entity3, Q1190554]: occurrence, [relation, P279]: subclass of, [entity4, Q10791]: nudity, [relation, P461]: opposite of, [entity5, Q11460]: clothing, [relation, P279]: subclass of, [entity6, Q14952]: headgear, [relation, P461]: opposite of, [entity5, Q11460]: clothing, [relation, P279]: subclass of,[entity6, Q14952]: headgear

Table 4: Created triples

Following, we searched for all entities' description on wikipedia website and extract 2-3 paragraphs from it to compose the paragraph part. Then we used each entity in each triple as starting point to take a random walk in all the triple sets that we extracted to create chains of triples with length 3-6 entities, see table 4.

In the end we put above data together in the order of sentence, triple, paragraph and triples with respect to each distinct triple we have to form the final dataset. Before tokenization we divided the dataset into training and testing sets using an 19:1 ratio. Thereafter, we proceeded to train the GPT2 model on the preprocessed dataset.

2.2 GPT2 model

GPT-2 model uses only the modified decoder module of Transformer model and is done in a multi-layer stacking fashion. The following figure[4] shows a schematic of the 12-layer GPT2 structure. Each layer is the same.

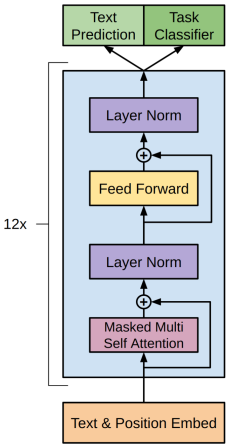

Figure 1: Basic structure of GPT2 model

The mechanism of GPT-2 is an "auto-regression" idea: after generating each token, it is added to the input sequence to form a new sequence. This new sequence then becomes the input to the model at the next time step.

At the beginning, we look for the embedding of the first token in the embedding matrix, and before passing this embedding to the first module of the model, it needs to be incorporated into the positional encoding, which indicates the order of the words in the sequence. The positional encoding matrix consists of a vector of positional encodings for each of the 1024 positions. Then, the first decoder-like module starts to process the token input embedding, first through the self-attention layer and then through the fully connected layer. After the first module processes the token, it gets a

result vector which is sent to the next module in next stack as input for processing. The processing is the same for each module, but each module has its own self-attention and neural network layers.

When the module at the top of the model produces an output vector (this vector is obtained after the Self Attention layer and the neural network layer), the model multiplies this vector by the embedding matrix.

The token with the highest score (top_k=1) can be chosen which is called greedy decoding. But if the model keep choosing the first top word, it may get into some strange loops. So a better strategy is to use the score as the probability of a word, and select a word from the whole list (so that the higher the score, the higher the chance of being selected). A compromise is to set top_k to 40 and have the model consider the 40 words with the highest scores,i.e., Beam Search. At this point, the model has completed one iteration and outputs one word. The model will continue to iterate according to the "auto-regressive" idea until all contexts have been generated (1024 tokens), or until tokens representing the end of the sentence have been output.

By using this mechanism, GPT2 model will learn the patterns, context and meaning from our specifically ordered data to generate textual knowledge graph (triples) base on the entities contained in input text.

2.2.1 Text and position embed

Segmentation in natural language processing refers to the process of slicing a text into words. Segmentation is an important basic task in the field of natural language processing, and its main purpose is to slice a continuous natural language text into sequences of words with certain semantic meanings, so as to facilitate further semantic analysis. For our input data we first need to use GPT2's byte pair encoding (BPE) tokenizer to transfer the input data into tokens. BPE tokenizer has its own dictionary and this dictionary records 50257 words and their serial numbers. Among them, "<|endoftext|>" is its last word and the index number of the last word is 50256.

The above processing procedure has converted the input text into a sequence of length 1024. The word vectors are further represented by a feature vector of each word and the length of the feature vector is the length of the hidden size. The number of words in the dictionary in the GPT2 model is 50257 and each word is represented by a 768 (hidden size) dimensional vector. This dictionary can be obtained by random generation and each word vector has been normalized with a normalized mean of 0 and variance of 1. Finally, a dictionary lookup is performed based on the indexed sequence numbers of the 1024 participles to obtain the word vectors of the corresponding sequence numbers.

In addition to word vector (text embedding), positional embedding is also needed to represent the position of a word as it appears in a sentence, which is very important for NLP. Therefore, GPT2 uses position embedding to save the relative or absolute position of the word in the sequence.

Position Embedding is denoted by PE, which has the same dimension as word embedding, and can be obtained by training or by using a certain formula. The latter is used in Transformer, and the formula[5] is as follows:

$$PE_{pos,2i} = sin(\frac{pos}{10000^{\frac{2i}{d}}})$$
$$PE_{pos,2i+1} = cos(\frac{pos}{10000^{\frac{2i}{d}}})$$

where pos denotes the position of the word in the sentence, d denotes the dimension of the PE (as in text embedding), 2i denotes the even dimension, and 2i+1 denotes the odd dimension (i.e., $2i \leq d$, $2i+1 \leq d$). Using this formulation to compute PE has the following benefits: Enables the PE to adapt to sentences that are longer than all the sentences inside the training set. Suppose the longest sentence inside the training set has 20 words, and suddenly a sentence of length 21 comes, the embedding of the 21st bit can be computed using the method of formula calculation. This allows the model to easily compute the relative position, and for a fixed length spacing k, PE(pos+k) can be computed using PE(pos). Since sin(A+B) = sin(A)cos(B) + cos(A)sin(B), cos(A+B) = cos(A)cos(B) - sin(A)sin(B).

Adding the text embedding and positional embedding of the word gives the representation vector x of the word, and x is the input to the model.

2.2.2 Masked multi self attention

In order to capture long-range dependencies and contextual relationships within our input text and be able to output triples only depending on the input text, we need masked multi self attention mechanism. Multi-Head Attention is composed of multiple Self-Attention. A normal self attention module allows a position to focus on the part to its right. A masked Self Attention module prevents the model from attending to future tokens in a sequence, ensuring that the predictions for a token only depend on the preceding tokens.

Self-attention mechanism means that the model will pay attention to the correlation between our input data when performing feature extraction. Attention indicates the amount of weight to focus on the data. Assuming that there is a set of sequences consisting of A, B, C, and D, then how much A is influenced by A, B, C, and D, respectively, corresponds to the degree of the attention, i.e., how much attention was paid to A, B, C, and D when extract the feature of A.

For this a QKV (Query, Key, Value) coding model was proposed in order to measure the value of such attention. Q, K, and V are equivalent to the way the original data is represented in the new feature space. The following figure[3][5] shows the structure of self-attention. In GPT2, self-attention receives input (matrix X consisting of word representation vectors x), and Q,K,V are obtained by linear transformation Wq, Wk, Wv of the input of self-attention. After obtaining the matrices Q, K, V, the output of self-attention can be calculated as follows[5]:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d^k}})V$$

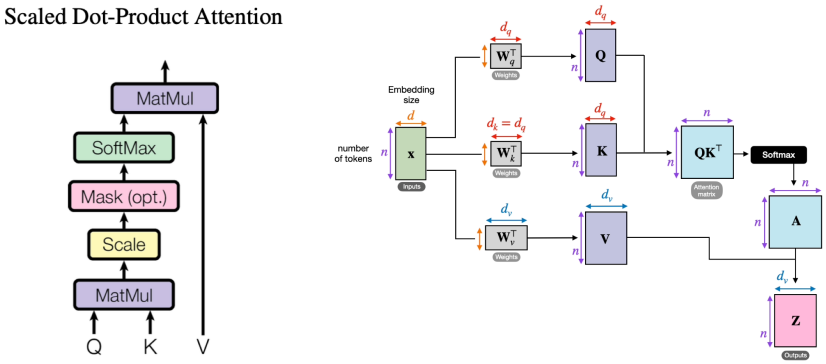

Figure 2: Structure of self-attention

Q is equivalent to defining a kind of query index of the original data, which can be analogized to the position of the elements in the original sequence. (K, V) represents the data value, the advantage of using this key-value way to represent the data is that the computation of the attention size does not affect the data value, i.e., the data value is decoupled from the attention size.

Attention size is represented by the correlation between Q and K. The larger the correlation, the larger the attention weight. As shown in the figure below, the correlation is computed by matrix multiplication, i.e., $Q \times K^T$. Each of its rows represents the correlation of the current element with all elements. In order to maintain the distributional properties of the output data, the multiplied data need to be divided by $\sqrt{d^k}$, with d^k being the output feature dimensions. Obviously, dividing by $\sqrt{d^k}$ does not change the relative size of the attention, but rather adjusts the overall magnitude. Attentional weights are multiplied with V, i.e., weighted summation, to obtain the outputs Z.

The multi-head attention mechanism is equivalent to employing the self-attention mechanism multiple times to extract the original features and stitch them together for fusion. We can also analogize the above self-attention process to a single convolution process, and the multi-head attention mechanism is equivalent to changing the number of output channels. If the attention module output dimensions and input dimensions are aligned, then the module can be stacked in multiple layers, i.e., the output of the previous module is used as the input of the subsequent module.

2.2.3 Norm and feed forward

Norm refers to layer normalization. Layer normalization will transform the inputs of each layer of neurons to have the same mean-variance, which can speed up the convergence. Equations for computing normalization:

Compute the Mean: x is the input vector of dimensionality d

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

Compute the Variance:

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

Normalize:ϵ is a small constant added for numerical stability to prevent division by zero

$$\hat{x_i} = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Scale and Shift: γ (scale) and β (shift) are learnable parameters that allow the normalized values to be scaled and shifted.

$$y_i = \gamma \hat{x_i} + \beta$$

Combining these steps, the final formula for Layer Normalization is:

$$y_i = \gamma \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

In Convolutional Neural Networks, for spliced features, a further layer of convolution or full link layer is usually used to fuse the spliced features as a whole. GPT2 also uses this kind of structure, i.e. Feed-Forward Network. The Feed Forward layer is relatively simple, it is a two-layer fully connected layer, the activation function of the first layer is Gaussian Error Linear Unit (Gelu), it can be expressed as:

$$GELU(x) = x\Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. A more practical approximation often used is:

$$GELU(x) = 0.5x(1 + tanh(\sqrt{\frac{\pi}{2}}(x + 0.044715x^3)))$$

and the second layer does not use the activation function, the corresponding Feed-Forward Network formula[5] is as follows:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

x is the input, and the Feed Forward ends up with an output matrix with the same dimension as x. The output matrix is the same as the input matrix of the Feed Forward layer.

2.2.4 Loss, optimizer and Metrics

For GPT2, the loss function is primarily used during the training process to guide the optimization of the model's parameters. In order to minimize the difference between its predicted entities and the actual entities in the training dataset, we use the typical loss function that used in GPT2: The cross-entropy loss. This loss function measures the difference between the probability distribution predicted by the model and the actual distribution. Mathematically, the cross-entropy loss for a single prediction is given by:

$$L = -\sum_i y_i log(p_i)$$

where $y_i$is the true distribution (1 for the correct word and 0 for all others) and p_i is the predicted probability for each word.

During training, the model generates predictions for the next word in a sequence. The cross-entropy loss is calculated for these predictions. The optimizer Adam uses this loss to update the model's parameters, aiming to reduce the loss over time.

For optimizer we use Adam (short for Adaptive Moment Estimation) optimizer to efficiently updating the model's parameters to minimize the loss function. And it is well-suited for handling large-scale neural networks and has become a standard choice for training transformer-based models.

During the training we also use metrics like recall, precision and F1-score to evaluate the model on evaluation dataset periodically.

3 Implementation

Our primary dataset originates from the 2015 Wikidata dump is downloaded from the Wikimedia Foundation `https://archive.org/details/wikimediadownloads`, which includes all entities and their respective IDs. We then randomly selected five relations and queried Wikidata to determine the corresponding entity for each relation, resulting in a set of triples. Next, we used the entity IDs to locate the corresponding Wikipedia page for each entity on Wikidata and extracted the textual information from these Wikipedia pages. Finally, we constructed hops in our dataset by searching for entities connected by five different relations to a depth of five.

3.1 Data extraction

During the data extraction process, we utilized various tools. In addition to the Wikimedia Foundation, the Wikidata Query Service and the Wikidata and Wikipedia APIs were also convenient and effective resources. Using the IDs obtained from the original entity dataset, we utilized the Wikidata Query Service to search for the names and IDs of other entities corresponding to specified relationships. The generated ternary sets are shown in Table 2.

To generate sentences corresponding to triples, we used the text generation capabilities of GPT-4o, which enables the creation of logical and human-readable sentences that accurately represent various relationships. Examples shown in table 3.

As for the paragraphs, the Wikipedia API has been proven to be useful for extracting the paragraphs we need. Basic idea here is the same as ternary generation. The entity IDs are also important as it links to a specific Wikidata page. Typically, a Wikidata page includes at least one version of the corresponding Wikipedia page in a particular language, most commonly the English version. From this Wikipedia page, we can extract several relevant paragraphs that provide detailed information and description about the entity. This approach ensures that we gather comprehensive and contextually rich descriptions for each entity in our dataset. In the end, all paragraphs with their corresponding entities and IDs are stored in a JSONL file, which can be considered a dictionary for the later merging task.

Entity IDs	paragraphs
Q8	Happiness is a positive and pleasant emotion, ranging from...
Q331769	In psychology, a mood is an affective state. In contrast...

Table 5: Examples of obtained paragraphs

We constructed hops by recursively traversing the entity-relation map. The process involved the following steps:

- **Initialization**: We initialized the hop construction with the starting entity and set the maximum traversal depth to five.
- **Traversal**: We traversed the entity-relation map, progressively and randomly building paths (hops) by following relations to connected entities. During traversal, we ensured that each entity was visited only once to prevent cycles.
- **Hop Formatting**: As we constructed the hops, we formatted them into a specific structure, detailing each entity and relation along the path. This structured representation facilitated easy interpretation and analysis of the hops.

See table 4 for more visual examples.

3.2 Data cleaning

So far we have fetched all the data we need. Nevertheless, these data remain separate from each other. Next we expect to merge them all together. Merging texts and triples is relatively easy, we just need to put the contents of the corresponding file with the same number of lines together. Correspondingly, there's a bit of a trick when it comes to merging paragraphs. For each pair of entity IDs in each triple, we retrieve the corresponding text from the JSON dictionary. If the entity ID is not found in the dictionary, the placeholder text "text not found" is used. Each subsequent line adds the retrieved text for the corresponding entity ID. These new lines combine the original line contents with the relevant entity descriptions and are compiled into a list.

In addition, when training a language model, a prefix label is usually added at the beginning in order for the model to understand the content of the text segment. This label serves to help the model better understand and process the text. In our case we added text, triple and hops to distinguish the four different parts of our dataset. We use the same prefixes since both text and paragraph are essentially text-based information. Also, in order to separate the four different parts of the dataset, we used the same format of separator as ChatGPT, e.g. <|text|> <|endoftext|> to denote the end of this part. The final format of our dataset can be visualized in the following figure 2.

KG1:
text2KG:
text: <|text|> happiness is an instance of mood. <|endoftext|>
triple: <|triple|> [entity1, Q8]: happiness, [relation, P31]: instance of, [entity2, Q331769]: mood <|endoftriple|>
text: <|text|> Happiness is a positive and pleasant emotion, ranging from… <|endoftext|>
KG:hops: <|triple|> [entity1, Q331769]: mood, [relation, P31]: instance of, [entity2, Q8]: happiness, [relation, P31]: instance of, [entity3, Q9415]: emotion <|endoftriple|>
KG2:
text2KG:
text: <|text|> A significant event for Universe is Big Bang. <|endoftext|>
triple: <|triple|> [entity1, Q1]: Universe, [relation, P793]: significant event, [entity2, Q323]: Big Bang <|endoftriple|>
text: <|text|> The universe is all of space and time and their contents. It comprises… <|endoftext|>
KG:hops: <|triple|>[entity1, Q1]: Universe, [relation, P793]: significant event, [entity2, Q323]: Big Bang, [relation, P31]: instance of, [entity3, Q1190554]: occurrence, [relation, P279]: subclass of, [entity4, Q10791]: nudity, [relation, P461]: opposite of, [entity5, Q11460]: clothing, [relation, P279]: subclass of, [entity6, Q14952]: headgear <|endoftriple|>

Figure 3: Format of input dataset

3.3 Training

Due to the large size of the dataset, training will take a large mount of time. So we use Horeka HPC systems from the Karlsruhe Institute of Technology, which can provide 76 cores of CPU or up to 4 A100 GPUs to accelerate the computation. The training language model is chosen as GPT2 with 124 million parameters.

To prepare our dataset for training a language model, we implemented a structured pipeline that processes and encodes text files efficiently. The process began by loading text files from a local directory using a custom function. Each file was read, tokenized using the GPT2 tokenizer from the tiktoken library, and appended with an end-of-text token.

To handle large volumes of data, we utilized parallel processing with 76 worker threads, significantly reducing processing time. The dataset was split into training (99.95%) and validation (0.05%) subsets which ensures a robust framework for model training and evaluation. The encoded data was then written to binary files in chunks to prevent memory overflow while using the np.uint16 data type for efficient storage.

The GPT-2 training loop is structured to facilitate gradient accumulation and mixed precision training. Gradient accumulation effectively simulates larger batch sizes by accumulating gradients across several mini-batches before updating the weights. Mixed precision training, utilizing PyTorch's automatic mixed precision (AMP) module, reduces memory usage and enhances computational efficiency. During the model's forward pass, token embeddings are computed, position embeddings are applied, and the sequence is processed through multiple transformer blocks. The resulting output logits are then used to calculate the cross-entropy loss against the target sequences. During the training the average loss and F1-score are used to evaluate the model periodically.

The default training configuration is shown below. After constructing the complete dataset in txt file format we used it directly as input to the GPT2 model. In general, one full training process lasts for 3 days. After that we also use 1 day to finetune the model at a low learning rate 0.00003.

Model	GPT-2
Drop out	0.0
Optimizer	AdamW
Learning rate	0.006-0.0006
Max. Iteration	600,000
Weight decay coefficient	0.1
Grad clip	1.0
Loss	Cross-Entropy Loss
Evaluation	Average loss & precision & recall & F1-Score

Table 6: Default Configuraion

3.4 Encountered issues

As the project progressed, some different problems occurred to us as well.

3.4.1 Excessive API requests

As previously mentioned, more than 11 million entities were involved in our dataset. When Wikidata and Wikipedia APIs are faced with a huge number of requests for access to the corresponding paragraphs, this can lead to limited access speeds and even program interruption errors.

Therefore, we came up with this efficient approach of using caching and batch in the query process to reduce the number of accesses to the API. The use of caching helps us to actively ignore previously queried data to avoid repeated accesses. And batch can help us package multiple data, by accessing the API once to query multiple data.

3.4.2 Encoding and Unicode Handling

Our dataset is not entirely clean and contains many special characters at first place after extraction from Wikidata, even though we selected English as the dataset language. During the lab, we encountered encoding issues, such as special letters like "è" with a hat on top, which are represented as Unicode escape characters. These display issues arise when using different compilers. To resolve this problem, we used the decoding method with unicode-escape, which automatically translates these Unicode characters into their respective special letters.

3.4.3 Memory Management during Data Preprocessing

It is normal to generate train.bin and val.bin files at the end of the final dataset preprocessing program. However, due to the large amount of data, the output of the bin files was interrupted by a memory error. To address this, we had to split the bin files into smaller chunks during preprocessing and then merged them together.

4 Results

Input	Expected Output	Actual Output
The sex or gender of Jack Bauer is male.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P21]: sex or gender, [entity2, Q6581097]: male <|endoftriple|>	triple:<|triple|> [entity1, Q16884225]: Jack Bauer, [relation, P21]: sex or gender, [entity2, Q6581097]: male <|endoftriple|>
Jack Bauer was educated at University of California, Berkeley.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P69]: educated at, [entity2, Q168756]: University of California, Berkeley	triple:<|triple|> [entity1, Q6111371]: Jack Bauer, [relation, P69]: educated at, [entity2, Q168756]: University of California, Berkeley
Jack Bauer received the award Silver Star.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P166]: award received, [entity2, Q852071]: Silver Star	triple:<|triple|> [entity1, Q6111442]: Jack Bauer, [relation, P166]: award received, [entity2, Q852071]: Silver Star

Table 7: Result of full dataset

In the results we compare the output obtained by training GPT-2 using the full dataset and only one-third of the dataset respectively, as shown in the table above. We can clearly see that the results of the models trained on the two different datasets are consistent in predicting the corresponding triples given the input text.

The difference between the two is also evident in the fact that the model trained with only one third of the dataset did not succeed in predicting the hops corresponding to the input text. The results of the model using the full dataset were able to accurately distinguish between entities and relationships and find matching ids. However, the model trained with only one third of the data performed poorly, sometimes failing to accurately identify the

Input	Expected Output	Actual Output
The sex or gender of Jack Bauer is male.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P21]: sex or gender, [entity2, Q6581097]: male <|endoftriple|>	triple:<|triple|> [entity1, Q6111301]: Jack Bauer, [relation, P21]: sex or gender, [entity2, Q6581097]: male <|endoftriple|>
Jack Bauer was educated at University of California, Berkeley.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P69]: educated at, [entity2, Q168756]: University of California, Berkeley	triple:<|triple|> [entity1, Q6524661]: Leon B. Garz, [relation, P69]: educated at, [entity2, Q168756]: University of California, Berkeley <|endoftriple|>
Jack Bauer received the award Silver Star.	triple:<|triple|> [entity1, Q24]: Jack Bauer, [relation, P166]: award received, [entity2, Q852071]: Silver Star	triple:<|triple|> [entity1, Q436584]: George C. Wilson, [relation, P166]: award received, [entity2, Q852071]: Silver Star <|endoftriple|>

Table 8: Result of 1/3 dataset

corresponding entities in the input text.

Table 9 below shows that as the amount of data increases, the performance of the model improves accordingly.

	precision	recall	F1-Score	loss
1/3 Dataset	0.865	0.819	0.734	0.58
Full Dataset	0.886	0.861	0.785	0.54

Table 9: Evaluation of two datasets

5 Conclusion

In this lab project, we first found a raw dataset that is suitable in all aspects for knowledge graph training. After that, we accomplished extensive data cleaning tasks to make it highly pure and well-organized in the requested format. It took many trials and errors to find the most appropriate training format that would allow the language model to learn each part of the dataset well. We then train the model on it with decaying learning rate, including a finetune process after the normal training cycle with adjusted low learning rate and obtained the desired results.

References

[1] karpathy, *nanoGPT*, GitHub repository Available: `https://github.com/karpathy/nanoGPT`. Last updated: 06.2024

[2] Wikimediadownlad `https://archive.org/details/wikimediadownloads`

[3] Sebastian Raschka. "Understanding and Coding the Self-Attention Mechanism of Large Language Models From Scratch" (2023) `https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html`

[4] Radford, Alec and Karthik Narasimhan. “Improving Language Understanding by Generative Pre-Training.” (2018).

[5] Vaswani, Ashish et al. “Attention is All you Need.” Neural Information Processing Systems (2017).